

End-to-End Social Media ETL Lab: Chronological Command-by-Command Runbook

Exact commands, script files, configuration steps, real-world troubleshooting, and verification

1. Lab Overview & Architecture

The social media data engineering lab implements an automated batch ETL pipeline designed to ingest, validate, score, load, and archive multi-modal social post data. The end-to-end flow is structured across three modular Airflow task groups:

- `extract_block`: Monitors local staging directories via a `FileSensor` (`wait_for_incoming_json`) and triggers parallel HDFS ingestion (`stage_raw_to_hdfs`).
- `transform_block`: Executes Python parsing routines (`run_python_transformation`) that perform image binary integrity checks (`validate_media_assets`) and engagement score calculations (`calculate_engagement_metrics`).
- `load_block`: Loads standardized 14-column records into SQLite (`load_to_sqlite`), snapshots the database state into cold HDFS archival storage (`archive_sqlite_to_hdfs`), and purges temporary landing files (`purge_local_staging`).

Airflow DAG Rendered Topology

The complete DAG layout as rendered in the Airflow Graph View:

None

```
[extract_block.wait_for_incoming_json] ->  
[extract_block.stage_raw_to_hdfs]
```

|
v

```
[transform_block.run_python_transformation]
```

|
v

```
[load_block.archive_sqlite_to_hdfs] --->  
[load_block.purge_local_staging]
```

2. Step 1: Environment & Directory Preparation

Activate the virtual environment and set mandatory Airflow variables inside the Ubuntu terminal:

Shell

```
source ~/airflow-venv/bin/activate
export AIRFLOW_HOME=~/.airflow
```

Construct the pipeline directory structure to handle incoming data, media assets, database storage, execution scripts, and DAGs:

Shell

```
mkdir -p ~/social-media-etl/data/incoming
mkdir -p ~/social-media-etl/data/media
mkdir -p ~/social-media-etl/database
mkdir -p ~/social-media-etl/scripts
mkdir -p ~/social-media-etl/dags
```

Generate initial mock JSON post payload and associated sample JPEG image assets for testing:

Shell

```
cat << 'EOF' >
~/social-media-etl/data/incoming/social_media_posts.json
[
  {
    "post_id": "POST001",
    "author_id": "user_alpha",
    "timestamp": "2024-03-15T10:30:00Z",
    "platform": "Twitter",
    "content_text": "Launching our new cloud pipeline!",
    "likes": 120,
    "shares": 45,
    "comments": 18,
    "impressions": 2500,
    "media_filename": "post001.jpg"
  },
  {
    "post_id": "POST002",
```

```

    "author_id": "user_beta",
    "timestamp": "2024-03-15T11:15:00Z",
    "platform": "Instagram",
    "content_text": "System architecture diagram breakdown.",
    "likes": 450,
    "shares": 89,
    "comments": 34,
    "impressions": 5800,
    "media_filename": "post002.jpg"
  },
  {
    "post_id": "POST003",
    "author_id": "user_gamma",
    "timestamp": "2024-03-15T12:00:00Z",
    "platform": "LinkedIn",
    "content_text": "Quarterly data engine performance review.",
    "likes": 85,
    "shares": 12,
    "comments": 5,
    "impressions": 1100,
    "media_filename": "post003.jpg"
  }
]
EOF

```

```

# Create sample image assets in the media directory
echo "fake image binary data 1" >
~/social-media-etl/data/media/post001.jpg
echo "fake image binary data 2" >
~/social-media-etl/data/media/post002.jpg
echo "fake image binary data 3" >
~/social-media-etl/data/media/post003.jpg

```

3. Step 2: Implementation of the ETL Transformation Engine

Deploy the Python transformation script. It handles JSON parsing, media validation, engagement calculations, schema recreation (with DROP TABLE IF EXISTS to prevent schema drift errors), and insertion of the full 14-column dataset into SQLite:

Python

```
cat << 'EOF' > ~/social-media-etl/scripts/process_posts.py
#!/usr/bin/env python3
import json
import os
import sys
import sqlite3
from PIL import Image

DB_PATH = os.path.expanduser("~/social-media-etl/database/social_analytics.db")
MEDIA_DIR = os.path.expanduser("~/social-media-etl/data/media")

def validate_media_assets(media_filename):
    if not media_filename:
        return False, "N/A", 0
    full_path = os.path.join(MEDIA_DIR, media_filename)
    if not os.path.exists(full_path):
        return False, "N/A", 0
    try:
        size_bytes = os.path.getsize(full_path)
        with Image.open(full_path) as img:
            width, height = img.size
            img.verify()
        return True, f"{width}x{height}", size_bytes
    except Exception:
        size_bytes = os.path.getsize(full_path) if os.path.exists(full_path) else 0
        return False, "N/A", size_bytes

def calculate_engagement_metrics(likes, shares, comments, impressions):
    if impressions == 0:
        return 0.0, 0.0
    weighted_score = likes + (shares * 2.0) + (comments * 1.5)
    engagement_rate = (weighted_score / impressions) * 100.0
    return round(weighted_score, 2), round(engagement_rate, 4)

def load_to_sqlite(json_filepath):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    # Drop existing table to ensure 14-column schema compliance
    cursor.execute("DROP TABLE IF EXISTS processed_posts;")
```

```

cursor.execute("""
    CREATE TABLE processed_posts (
        post_id TEXT PRIMARY KEY,
        author_id TEXT NOT NULL,
        timestamp TEXT NOT NULL,
        platform TEXT NOT NULL,
        content_text TEXT,
        likes INTEGER,
        shares INTEGER,
        comments INTEGER,
        impressions INTEGER,
        weighted_engagement REAL,
        engagement_rate REAL,
        media_valid INTEGER,
        media_dimensions TEXT,
        media_size_bytes INTEGER
    );
""")

with open(json_filepath, 'r') as f:
    posts = json.load(f)

for post in posts:
    p_id = post.get("post_id")
    author = post.get("author_id")
    ts = post.get("timestamp")
    platform = post.get("platform")
    text = post.get("content_text", "")
    likes = int(post.get("likes", 0))
    shares = int(post.get("shares", 0))
    comments = int(post.get("comments", 0))
    impressions = int(post.get("impressions", 1))

    is_valid, dims, size_bytes = validate_media_assets(post.get("media_filename"))
    weighted_score, eng_rate = calculate_engagement_metrics(likes, shares,
comments, impressions)

    cursor.execute("""
        INSERT INTO processed_posts VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            p_id, author, ts, platform, text,

```

```

        likes, shares, comments, impressions,
        weighted_score, eng_rate,
        1 if is_valid else 0, dims, size_bytes
    ))

    conn.commit()
    conn.close()
    print(f"Successfully processed {len(posts)} posts into SQLite.")

if __name__ == "__main__":
    if len(sys.argv) > 1:
        load_to_sqlite(sys.argv[1])
EOF

```

4. Step 3: Implementation of the Airflow Modular DAG

Construct the modular DAG definition file using TaskGroup structures for extract_block, transform_block, and load_block:

```

Python
cat << 'EOF' > ~/social-media-etl/dags/social_media_etl_dag.py
from datetime import datetime, timedelta
import os
from airflow import DAG
from airflow.sensors.filesystem import FileSensor
from airflow.operators.bash import BashOperator
from airflow.utils.task_group import TaskGroup

DEFAULT_ARGS = {
    'owner': 'data_engineering',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=2),
}

INCOMING_DIR = os.path.expanduser("~/social-media-etl/data/incoming")
SCRIPT_PATH = os.path.expanduser("~/social-media-etl/scripts/process_posts.py")

```

```
DB_PATH = os.path.expanduser("~/social-media-etl/database/social_analytics.db")
```

```
with DAG(  
    'social_media_etl_pipeline',  
    default_args=DEFAULT_ARGS,  
    description='End-to-End Social Media ETL Processing Engine',  
    schedule_interval='@hourly',  
    catchup=False,  
    max_active_runs=1,  
) as dag:
```

```
    with TaskGroup("extract_block") as extract_block:
```

```
        wait_for_file = FileSensor(  
            task_id='wait_for_incoming_json',  
            filepath=INCOMING_DIR,  
            fs_conn_id='fs_default',  
            poke_interval=10,  
            timeout=300,  
        )  
        stage_to_hdfs = BashOperator(  
            task_id='stage_raw_to_hdfs',  
            bash_command=(  
                "hdfs dfs -mkdir -p /user/hadoop/social_media/raw/ && "  
                "hdfs dfs -put -f ~/social-media-etl/data/incoming/*.json  
/user/hadoop/social_media/raw/"  
            ),  
        )  
        wait_for_file >> stage_to_hdfs
```

```
    with TaskGroup("transform_block") as transform_block:
```

```
        execute_processing = BashOperator(  
            task_id='run_python_transformation',  
            bash_command=(  
                "for f in ~/social-media-etl/data/incoming/*.json; do "  
                " [ -f \"$f\" ] || continue; "  
                f" python3 {SCRIPT_PATH} \"$f\"; "  
                "done"  
            ),  
        )
```

```
    with TaskGroup("load_block") as load_block:
```

```

archive_database = BashOperator(
    task_id='archive_sqlite_to_hdfs',
    bash_command=(
        "TS=$(date +%Y%m%d_%H%M%S) && "
        "hdfs dfs -mkdir -p /user/hadoop/social_media/archive/ && "
        f"hdfs dfs -put {DB_PATH}"
        "/user/hadoop/social_media/archive/db_snapshot_${TS}.db"
    ),
)
purge_local_incoming = BashOperator(
    task_id='purge_local_staging',
    bash_command="rm -f ~/social-media-etl/data/incoming/*.json",
)
archive_database >> purge_local_incoming

extract_block >> transform_block >> load_block
EOF

```

5. Step 4: Connecting Host Mac Chrome to Airflow UI

To access the Airflow Web UI from the macOS host browser, establish an SSH port forwarding tunnel. CRITICAL: Execute this command strictly in the Mac host Terminal (not inside the VM instance):

```

Shell
ssh -p 2222 -N -L 8080:localhost:8080 athu@127.0.0.1

```

Once the tunnel is open, access the Airflow interface in Google Chrome on your host Mac:

- URL: `http://localhost:8080`
- **Credentials:** `admin / admin`

6. Step 5: Triggering and Monitoring the Pipeline

Execute the following commands inside the Ubuntu VM terminal to unpause, trigger, and inspect the state of the pipeline run:

```

Shell
# Unpause and trigger the DAG execution
airflow dags unpause social_media_etl_pipeline
airflow dags trigger social_media_etl_pipeline

```

```
# List all runs for the DAG
airflow dags list-runs -d social_media_etl_pipeline

# Check individual task state breakdown using the DAG run ID
airflow tasks states-for-dag-run social_media_etl_pipeline
"manual__2024-03-15T12:00:00+00:00"
```

7. Step 6: Step-by-Step Troubleshooting Log (Errors Encountered & Fixes Applied)

Error 1: ERR_CONNECTION_REFUSED on Mac Host Chrome Browser

- Command Executed: Running `ssh -p 2222 -N -L 8080:localhost:8080 athu@127.0.0.1` directly inside the Ubuntu VM terminal shell.
- Symptom / Error: `ssh: connect to host 127.0.0.1 port 2222: Connection refused`
- **Root Cause:** Port 2222 is forwarded on the Mac host system to map into the VM's internal SSH server (listening on port 22). Running the SSH command inside the guest VM tries to connect to the VM itself on port 2222, which is not open.
- Fix Applied: Open a native Mac Terminal window and execute the SSH port forwarding command from the host side.

Error 2: Command 'sqlite3' Not Found

- Command Executed: Executing `sqlite3 ~/social-media-etl/database/social_analytics.db` directly in the bash terminal.
- Symptom / Error: Command 'sqlite3' not found, but can be installed with: `sudo apt install sqlite3`
- **Root Cause:** The standalone native C CLI binary (`sqlite3`) was omitted from the base server installation, though Python's standard library includes internal SQLite bindings.
- Fix Applied: Query database records directly via Python standard library inline commands without requiring external binary installations:

```
Shell
python3 -c "import sqlite3;
conn=sqlite3.connect('/home/athu/social-media-etl/database/social_analytics.db');
print(conn.execute('SELECT * FROM processed_posts;').fetchall()); conn.close()"
```

Error 3: Schema Drift Mismatch (load_block Failure)

- Command Executed: Execution of transform_block.run_python_transformation task group.
- Symptom / Error: sqlite3.OperationalError: table processed_posts has 13 columns but 14 values were supplied
- **Root Cause:** The SQLite table was initially initialized with 13 columns. When the `media_size_bytes` metric was introduced later, `CREATE TABLE IF NOT EXISTS` ignored the new field, causing an operational schema mismatch during batch insertion.
- Fix Applied: Updated process_posts.py to execute DROP TABLE IF EXISTS processed_posts; prior to table creation, guaranteeing an aligned 14-column layout.

Error 4: ValueError: year 0 is out of range During CLI Status Check

- Command Executed: airflow tasks states-for-dag-run social_media_etl_pipeline manual__2024-03-15T12:00:00
- **Symptom / Error:** `ValueError: year 0 is out of range` emitted from Pendulum date library.
- Root Cause: Unquoted ISO timestamp identifiers passed into CLI parsing logic get stripped or misinterpreted by regular expressions, causing Pendulum to receive a zeroed timestamp parameter.
- Fix Applied: Pass exact quoted DAG run ID string parameters or retrieve task run states programmatically using Python API calls.

8. Step 7: Final Data Validation & Proof of Execution

To confirm pipeline execution, inspect SQLite database records and HDFS storage directories:

-

```
Shell
# Query target records from SQLite database
python3 -c "
import sqlite3
conn = sqlite3.connect('/home/athu/social-media-etl/database/social_analytics.db')
cursor = conn.cursor()
cursor.execute('SELECT post_id, platform, weighted_engagement, engagement_rate,
media_dimensions, media_size_bytes FROM processed_posts;')
for row in cursor.fetchall():
    print(row)
conn.close()
"
```

```
# Verify raw landed datasets in HDFS
```

```
hdfs dfs -ls -h /user/hadoop/social_media/raw/
```

```
# Verify cold database archive snapshots in HDFS
```

```
hdfs dfs -ls -h /user/hadoop/social_media/archive/
```

Airflow UI Final Graph Verification

In the Airflow Graph View, verify that all 7 tasks across the 3 TaskGroups (extract_block.wait_for_incoming_json, extract_block.stage_raw_to_hdfs, transform_block.run_python_transformation, load_block.archive_sqlite_to_hdfs, and load_block.purge_local_staging) display a dark green state border, confirming a successful end-to-end execution run.